

Hermes Agent: Project Architecture & Execution Plan

The Hermes Agent is a fully automated, agentic pipeline designed to aggregate, evaluate, and publish high-quality "good first issues" from GitHub. Leveraging LangGraph, the system intelligently scores open-source issues based on their alignment with specific technology stacks and automatically generates a weekly curated LinkedIn post featuring the top 5 opportunities for the community.

1. Technology Stack & Infrastructure

- **Core Framework:** Python utilizing LangGraph (for state management) and LangChain.
- **Intelligence:** LLM (OpenAI, Anthropic, or Gemini) for issue evaluation, scoring, and social media copywriting.
- **Automation Pipeline:** GitHub Actions (utilizing cron jobs for zero-cost execution).
- **Database:** MongoDB Atlas (NoSQL DB provisioned via the GitHub Student Developer Pack).
- **Integrations:** GitHub REST API / MCP (for data scraping) and LinkedIn API (for content publishing).

2. System Architecture

Phase 1: Daily Scrape & Score

A GitHub Action triggers the LangGraph agent daily. The Scraper Node fetches recent issues from targeted repositories. The raw data is passed to the Evaluator Node, where the LLM parses the issue description, assigns a quality score (1-100), and filters out irrelevant or overly complex tasks.

Phase 2: Database Storage

Evaluated issues are inserted directly into MongoDB Atlas. The JSON schema includes the issue metadata (title, URL, repository), the LLM-generated score, the targeted tech stack, and a boolean flag (is_published: false) to ensure issues are not posted twice.

Phase 3: Weekly Aggregation & Publishing

Every Friday morning, a secondary GitHub Action executes. It queries MongoDB for the top 5 highest-scoring, unpublished issues from the past week. A Generator Node drafts a structured LinkedIn post highlighting the technical benefits of tackling each issue, then pushes the content live via the LinkedIn API. The database records are then updated to `is_published: true`.

3. LLM Scoring Matrix (1-100 Scale)

Evaluation Criteria	Weight	Description
Tech Stack Match	40%	Strict alignment with core technologies (e.g., React, Vite, Tailwind CSS, Java, Spring Boot, FastAPI, MySQL).
Issue Clarity	30%	Presence of a clear description, reproducible steps, and well-defined acceptance criteria.
Setup Difficulty	20%	Availability of comprehensive contributing guidelines and simple local environment setup.
Repository Activity	10%	Evidence of recent commits, active maintainers, and responsive PR reviews.

4. Execution Steps

1. **Database Provisioning:** Activate the MongoDB Atlas offer, create an M0 cluster, and secure the connection string.
2. **API Setup:** Generate standard GitHub Personal Access Tokens and configure the LinkedIn Developer App for OAuth token generation.
3. **Agent Development:** Write the LangGraph workflow, focusing extensively on the system prompts for the Evaluator Node.
4. **Pipeline Automation:** Create the YAML workflows in GitHub Actions, setting up the daily cron job for scraping and the weekly cron job for publishing.

5. Implementation Blueprint (Antigravity Scaffolding)

This section provides the strict technical constraints required for the Antigravity environment to orchestrate and scaffold the codebase without hallucination.

5.1 Directory Structure

```
hermes-agent/
├── .github/
│   └── workflows/
│       ├── daily_scrape.yml
│       └── weekly_publish.yml
├── src/
│   ├── agents/
│   │   ├── graph.py    # LangGraph state machine definition
│   │   ├── nodes.py    # Node functions (Scraper, Evaluator, Generator)
│   │   └── state.py     # TypedDict state definitions
│   ├── database/
│   │   └── mongo_client.py # MongoDB Atlas connection and operations
│   └── utils/
│       ├── github_api.py # REST/MCP integration for GitHub
│       └── linkedin_api.py # OAuth and posting logic
├── requirements.txt
└── main.py
```

5.2 LangGraph State Schema

```
from typing import TypedDict, List
```

```
from pydantic import BaseModel
```

```
class Issue(BaseModel):
```

```
    id: str
```

```
    title: str
```

```
    url: str
```

```
    description: str
```

```
    labels: List[str]
```

```
    score: int = 0
```

```
    is_published: bool = False
```

```
class AgentState(TypedDict):
```

```
    target_repos: List[str]
```

```
    raw_issues: List[dict]
```

```
    evaluated_issues: List[Issue]
```

```
    post_draft: str
```

5.3 Database Schema (MongoDB JSON)

```
{
  "_id": "ObjectId",
  "issue_id": "123456789",
  "title": "Fix React hook dependency array",
  "url": "https://github.com/repo/issues/12",
  "repository": "owner/repo",
  "tech_stack": ["React"],
  "evaluation_score": 85,
  "scraped_at": "2026-07-30T23:54:05Z",
  "is_published": false
}
```

5.4 Explicit Dependencies

```
langgraph==0.0.X
```

```
langchain-core==0.1.X
```

```
langchain-openai==0.0.X
```

motor==3.3.X

PyGithub==2.X

pydantic==2.X